

Island Escape: A Multi-Agent Survival Trading Game with LLM Negotiation

Yufeng He and Team

May 2026

Abstract

Island Escape is a 2D survival trading game built around a simple question: what happens when resource management is no longer only a numerical puzzle, but also a social negotiation problem? The player is stranded on an island with four AI-controlled characters. Every day, each character must gather food, decide whether to sell resources to a merchant ship, negotiate with other islanders, and survive the nightly resource drain. The first character to reach 100 coins escapes.

Our main contribution is not just that we placed a language model inside a game. The more important engineering decision was to keep the game state deterministic and auditable while allowing the language model to influence high-level intentions, dialogue, and bargaining behavior. The server remains the source of truth. The LLM proposes actions and natural-language responses, but all resource updates, phase transitions, trade execution, and win/loss checks are validated by typed game logic. This gives us a game that feels social and emergent without handing correctness to an unpredictable model.

During the final development stage, we extended the project with daily random events, a day-scaled boss dungeon challenge, improved the negotiation interface, clearer phase and resource feedback, and tighter validation across linting, type checking, tests, and builds. The resulting prototype demonstrates an integrated game loop: movement, resource labor, LLM-based negotiation, persistent game state, real-time AI turn events, and an optional action-combat challenge that creates a stronger final demo arc.

1 Project Vision

Island Escape began as a small survival economy, but the design target was larger than a resource spreadsheet. We wanted a game where the interesting decisions come from the interaction between three forces: scarcity, opportunity, and social pressure. Scarcity comes from nightly fish and wheat consumption. Opportunity comes from the merchant ship’s fluctuating prices. Social pressure comes from NPCs who have their own resources, incentives, and personalities.

The player is not simply optimizing a fixed formula. They need to decide when to gather, when to sell, when to bargain, and when to take risks. The AI agents turn that decision space into a more human-feeling negotiation game: Tom may be cautious, Lily may cooperate, Sam may trade aggressively, and Jack may exploit weakness. In practice, this makes the same mechanical state feel different depending on who the player approaches and what they say.

My role in the project was to keep the design ambitious while making the implementation robust enough for a live demo. I treated the LLM as an unreliable collaborator rather than as a trusted database. That framing shaped most of the architecture: typed shared schemas, server-side validation, deterministic state transitions, guarded negotiation flows, and a UI that gives the player immediate feedback when the model is thinking.

2 Design Goals

2.1 A readable game loop

The core loop needed to be understandable within a short presentation. We therefore used a daily rhythm:

1. the day starts, merchant prices refresh, and a daily event is rolled;
2. the player must perform labor, either fishing or farming;
3. the player spends up to two trade slots;
4. AI agents take their turns;
5. settlement consumes resources and checks elimination or escape.

This structure gives the game a clean tactical cadence. The player always knows what kind of decision they are making: survival first, trade second, then consequences. Daily events add controlled variation without changing the phase order.

2.2 LLM agents with constrained authority

The language model should make the characters feel alive, but it should not be able to corrupt the game. In Island Escape, the model can decide what an NPC wants to do and what it says, but it cannot directly mutate coins, fish, wheat, friendship, or phase state. All of those updates pass through server-side game engine functions.

This is an important engineering distinction. A naive LLM game often lets the model narrate facts into existence. We avoided that. The model can propose a trade. The server validates whether the resources exist. The model can accept a proposal. The server decides which concrete proposal is executed. The model can produce colorful text. The state machine decides whether the game advances.

2.3 Presentation-first polish

The final stage focused on making the demo reliable. In a classroom presentation, a clever system is less convincing if the audience cannot see what is happening. We therefore prioritized visible phase guidance, immediate optimistic chat feedback while waiting for an LLM response, thinking states for NPC dialogue, resource-change feedback, a boss dungeon for a more dramatic demo moment, and validation commands that can be rerun before submission.

3 Gameplay System

3.1 Resources and win condition

Each character has fish, wheat, coins, trade slots, and status flags. Fish and wheat are survival resources. Coins are the escape resource. The win condition is deliberately simple: reach 100 coins and escape. The difficulty comes from the fact that coins are not gathered directly. They must be obtained through selling resources or by taking the risk of the dungeon reward.

Fishing gives immediate fish. Farming creates a delayed wheat harvest. This small asymmetry creates a useful tradeoff: the player can stabilize today or invest in a later day. Because every character consumes resources at settlement, running out of either fish or wheat eliminates them.

Resource	Role	Main source
Fish	Immediate survival resource	Fishing labor
Wheat	Delayed survival resource	Farming and harvest queue
Coins	Win-condition resource	Merchant sale, dungeon reward
Trade slots	Daily action budget	Reset each day

Table 1: Core resources in Island Escape.

3.2 Trade and negotiation

Trading has two forms. The merchant ship is deterministic: the player sells fish or wheat at the current day’s prices. NPC negotiation is social: the player sends a message and optionally attaches a structured proposal. The NPC responds through the LLM, possibly with an offer, counter-offer, acceptance, or rejection.

The key design choice is that natural language and structured trade proposals coexist. The text makes the interaction expressive. The structured proposal makes it executable. If a player says, “I can give you fish,” that may be flavorful, but the server still needs exact amounts before resources can move. This keeps the negotiation interface flexible without sacrificing correctness.

3.3 Daily constraints

Several constraints prevent trivial or broken play:

- the player must labor before trading;
- trade actions only run during the trade phase;
- each day has a limited number of trade slots;
- a player cannot repeatedly trade with the same NPC in the same day;
- an active negotiation must be completed or closed before another starts;
- dungeon entry consumes a trade slot and is limited to once per day.

These checks are implemented server-side, not just in UI buttons. That matters because the frontend can be bypassed, refreshed, or delayed by LLM calls. The server is the authority.

3.4 Daily random events

The final gameplay pass adds a daily event system to make repeated days less predictable. The event is stored in shared game state as **dailyEvent**, rolled at dawn, written into the day-start log, and shown in the HUD as a compact badge. The first two days are always calm so new players can learn the loop before modifiers appear. Later days are still often calm, but roughly one third introduce a rule change. The most punishing event, drought, is suppressed until day 4 so early runs are not decided before players have wheat buffers.

These events are implemented inside deterministic game-engine functions rather than by the LLM. For example, storm changes the result of fishing labor, festival changes the friendship bonus after a peer trade, lucky catch grants fish during day start, and drought changes the settlement cost. This keeps event play visible and varied while preserving the same server-authoritative rule model as the rest of the economy.

Event	Gameplay effect	Design role
Calm day	Standard rules	Baseline economy
Storm	Fishing yields 2 fish instead of 3	Pushes scarcity and trade
Festival	Peer-trade friendship gain is doubled	Rewards social play
Lucky Catch	Alive characters gain 2 fish at dawn	Recovery and tempo boost
Drought	Night upkeep costs 2 wheat	Punishes weak farming plans

Table 2: Daily event variants added to the core economy.

4 AI Agent Design

4.1 Agent responsibilities

The AI layer has two different jobs. First, AI agents take turns during the simulation: they choose labor and trade actions. Second, NPCs respond during negotiations with the player or with other AI agents. These are related but not identical tasks.

For turn decisions, the model receives a state summary and produces a structured decision: labor plus up to two trade choices. For negotiation, the model receives the character context, the partner, current resources, personality, and dialogue history. It then returns a short response and, when appropriate, a concrete offer/request pair.

4.2 Prompting philosophy

The prompts were written to make the model behave like a game character rather than a generic assistant. The system message asks for role-consistent behavior, but the server still checks the result. This gives each NPC a different behavioral tendency:

- Tom is cautious and resource-protective.
- Sam is a fast, aggressive trader.
- Lily is cooperative and alliance-oriented.
- Jack is opportunistic and persuasive.

The important part is that these personalities affect negotiation style without overriding the economy. If Jack wants to exploit the player, he still has to make an executable trade.

4.3 Model and provider boundary

The project uses an OpenAI-compatible client. The current development configuration is designed for DeepSeek through OpenRouter, using a model name such as `deepseek/deepseek-chat`. This is intentionally abstracted behind environment variables: `OPENAI_API_KEY`, `OPENAI_BASE_URL`, and `OPENAI_MODEL`.

No real key belongs in the repository, report, slides, or screenshots. This is both a security practice and a reproducibility practice: the code should explain what variables are needed, but not leak a working credential.

4.4 Handling LLM latency

One final-demo issue we identified was that a player could send an NPC message and not immediately see feedback. Because the LLM call may take a few seconds, that makes the UI feel frozen even when the request is working. The frontend now uses optimistic chat display and a pending/thinking state. The player's message appears immediately, then the authoritative server response replaces the local message list when it returns.

This is a small interaction detail, but it matters. Without it, the audience may interpret model latency as a broken game. With it, latency becomes visible as part of the AI interaction.

5 Software Architecture

5.1 Monorepo layout

The project is organized as a TypeScript monorepo:

- **apps/web**: Vue 3 frontend, PixiJS canvas, 3D preview, UI state.
- **apps/server**: Fastify backend, game routes, LLM agent runtime, SQLite persistence.
- **packages/shared**: Zod schemas and shared TypeScript types.

This layout prevents the frontend and backend from drifting apart. Player actions, game state, daily events, SSE events, and dungeon results are all described in shared schemas. The server validates incoming actions against those schemas, while the frontend receives inferred TypeScript types.

5.2 Server as source of truth

The backend owns the game state. The frontend sends actions to `/api/games/:id/action`; the backend validates the action, updates the state, persists it, and returns the new state. During AI turns, the backend also broadcasts SSE events so the frontend can show progress without polling.

This design is deliberately conservative. In a game involving asynchronous LLM calls, local UI state can easily become stale. By keeping the server authoritative, we avoid confusing cases where the UI thinks a trade happened but the resource ledger disagrees.

5.3 State machine discipline

Most game mechanics are implemented as explicit phase transitions:

- **player_labor**: only fishing or farming is valid.
- **player_trade**: merchant trade, NPC negotiation, dungeon entry, or ending the turn is valid.
- **ai_turns**: AI agents take labor/trade actions.
- **settlement**: resources are consumed and victory/elimination is checked.

This prevents ambiguous behavior. For example, an NPC negotiation should not start during settlement. Dungeon results should not apply outside the dungeon flow. The phase model makes these cases straightforward to reject.

5.4 Persistence and real-time events

Game sessions are kept in memory for active play and persisted to SQLite through Drizzle. The frontend opens an SSE stream for each game. Events include state updates carrying the daily event state, AI thinking, AI decisions, negotiation lines, trade results, settlement events, eliminations, escapes, and dungeon events.

This event model makes the game easier to present. The audience can see that AI agents are not silent background calculations; they are taking turns, making trades, and affecting the state.

6 Boss Dungeon Extension

The boss dungeon was added as a final-stage gameplay extension. The original survival/trading loop is strategic but slow. For a short demo, we wanted one high-energy moment that still connects to the economy. The dungeon does that.

Mechanically, the dungeon is an optional risk/reward action during the trade phase:

- entering costs one trade slot;
- only one run is allowed per day;
- winning grants a day-scaled coin reward;
- losing removes some fish and wheat;
- the player returns to the island flow afterward.

Inside the dungeon, the player fights a boss using movement, bullets, flash movement, an ultimate skill, XP orbs, and upgrade cards. This gives the game a second mode without replacing the main economy. The boss fight is not a separate game pasted on top; it is another way to make an economic decision under risk.

The later boss update connects combat difficulty to the island day. When the frontend enters dungeon mode, it passes the current day into `DungeonArena`. The arena stores that value as a day level and passes it to `Boss.setDifficulty`. Boss maximum HP increases by 50 per day, bullet and charge damage increase by one per day with caps, attack cooldowns shrink toward a lower bound, and minion spawn pacing, minion caps, and minion health also scale upward. Clearing the dungeon also scales the reward by adding 4 coins for each day after day 1, capped at 80 coins. This creates a real timing tradeoff: entering early is safer but pays less, while delaying the dungeon can accelerate escape but makes the fight more dangerous.

7 Visual Rendering and Animation

The final visual pass keeps the playable island and dungeon in PixiJS, while a separate Three.js canvas renders the left-side interaction preview. This hybrid approach improves presentation quality without moving tile logic, collision checks, phase control, or server-authoritative game state into a 3D scene.

The preview reads the same interaction state as the action menu. It shows a default island when no object is nearby, then swaps to procedural low-poly models for fishing, farmland, merchant trading, NPC dialogue, and dungeon entry. Each model rotates and floats subtly. The Three.js renderer caps device pixel ratio, resizes with `ResizeObserver`, pauses when the document is hidden, and disposes old geometry and materials when the active model changes.

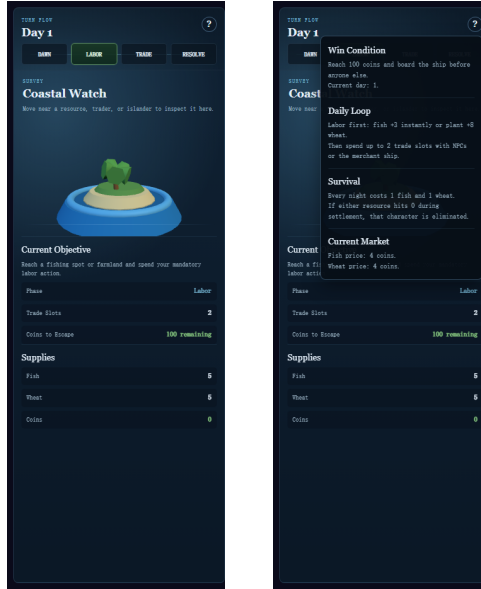


Figure 1: Left interaction panel in its default state and with the rules popover open. The compact horizontal phase flow leaves more room for the 3D preview and state-specific parameters.

The side panel now works as a contextual inspector rather than a decorative preview. Fishing, farming, merchant, NPC, and dungeon states expose their relevant parameters, such as yield, labor cost, harvest timing, market prices, friendship, resources, and combat entry constraints. Longer phase and rules descriptions are moved into hover popovers so the default layout stays compact. The preview canvas has a 300-pixel minimum width, and popovers use a higher stacking order so explanations remain visible above the model and parameter grid.

Character labels were moved into a top PixiJS overlay layer rendered after the tile, character, and effects layers. The labels use higher-resolution text, stronger strokes, extra texture padding, and rounded pixel coordinates. This keeps NPC names readable over terrain and animation effects.

The dungeon reuses the same PixiJS application but switches from the island container to **DungeonArena**. Its animation layer includes player combat, boss states, pooled bullets, minions, XP orbs, hit sparks, damage numbers, screen shake, hit-stop, flashes, vignette, and attack telegraphs. Browser screenshots and frontend checks confirmed that the PixiJS/Three.js split made the interface richer while preserving readable gameplay.

8 Testing and Validation

The project uses several validation layers:

- schema tests in `packages/shared`;
- backend API and game-flow tests in `apps/server`;
- frontend unit tests for UI-derived state;
- TypeScript checks across the workspace;
- lint checks for frontend quality;
- production build validation through Vite and TypeScript.



Figure 2: Main canvas after the label-layer update. NPC names are rendered above the world and remain sharp enough to identify characters during negotiation and movement.

Command	Purpose	Expected final status
<code>pnpm lint</code>	static quality checks	Pass
<code>pnpm typecheck</code>	TypeScript correctness	Pass
<code>pnpm test</code>	unit/API/schema tests	Pass
<code>pnpm build</code>	production build	Pass with possible chunk-size warning

Table 3: Final validation checklist.

One useful lesson from final integration was that passing unit tests alone was not enough. After merging teammate work, the test suite passed, but typecheck and build initially caught integration issues in shared state fields and helper functions. Fixing those issues before submission improved the reliability of the final demo and made the repository easier for teammates to continue.

9 Team Contributions

The final project was collaborative, but the work was not evenly distributed at every stage. My contribution was strongest in system integration, AI architecture, final stabilization, and preparing the materials needed for a credible presentation.

- **Yufeng He:** project coordination, report draft, presentation narrative, LLM/agent architecture direction, final integration, validation workflow, and demo-readiness checks.
- **Developer 1:** dialogue-related bug fixes and final PPT preparation based on the project proposal and new progress.
- **Erfei Yu:** daily event gameplay, boss dungeon feature work, day-scaled boss difficulty, flow polishing, and demo video recording/editing.
- **Zhiyuan:** visual rendering, animation, and 3D UI technical explanation for the report.

From an engineering perspective, the most important coordination decision was to separate responsibilities late in the project. At that point, adding broad new features would have been risky. We focused on a stable demo path: NPC dialogue, resource/trade feedback, boss dungeon, report, PPT, and video.

10 Limitations

Island Escape is a prototype, and several limitations are still visible:

- **LLM latency:** NPC responses can take several seconds depending on provider speed and network conditions.
- **Model variability:** different model outputs can change the tone and quality of negotiation, even under the same game state.
- **Balance:** daily events, dungeon scaling, merchant prices, and survival costs may need more playtesting.
- **Presentation risk:** a full game can be slow, so the final demo should combine a live short path with an edited video.

- **Visual asset depth:** the current 3D preview models are procedural, so authored models and textures could further improve fidelity.

11 Future Work

The most interesting future direction is to make the AI agents remember social history more deeply. The current game has friendship and dialogue history inside negotiations, but a richer version could track promises, betrayal, past prices, and alliance formation across days.

Another direction is evaluation. Because LLM-driven games are partly subjective, we should not only test code correctness. We should also measure whether players understand the phase flow, whether negotiations feel fair, whether NPC personalities are distinguishable, and whether the boss dungeon improves the demo without distracting from the main AI-agent concept.

Finally, the dungeon and economy could be coupled more strongly. For example, different NPC relationships could unlock cards, boss fights could change merchant prices, or AI agents could react socially if the player repeatedly chooses combat over cooperation.

12 Conclusion

Island Escape demonstrates how a small game can use LLM agents in a disciplined way. The language model adds personality, negotiation, and emergence; the state machine preserves correctness. That separation is the core engineering lesson of the project.

The final prototype combines a readable survival economy, daily random events, AI-driven NPC negotiation, real-time event updates, a polished interaction UI, and a day-scaled boss dungeon that gives the presentation a stronger climax. More importantly, the project shows a practical pattern for future AI games: let the model create intent and expression, but let typed code own the rules.

References

- [1] Joon Sung Park et al. *Generative Agents: Interactive Simulacra of Human Behavior*. 2023. <https://arxiv.org/abs/2304.03442>
- [2] a16z-infra. *AI Town*. <https://github.com/a16z-infra/ai-town>
- [3] Project Sid. *Multi-Agent AI Simulation Reference*. 2024.